1. Persiapan Data

Import library yang diperlukan

Pada bagian ini kita mengunduh library yang akan digunakan untuk memantau penggunaan CPU dan GPU saat melakukan training data

In [25]:

```
pip install psutil GPUtil
```

```
Requirement already satisfied: psutil in /usr/local/lib/python3.11/dist-packages (5.9.5)
Collecting GPUtil
  Downloading GPUtil-1.4.0.tar.gz (5.5 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: GPUtil
  Building wheel for GPUtil (setup.py) ... done
  Created wheel for GPUtil: filename=GPUtil-1.4.0-py3-none-any.whl size=7392 sha256=3402ad88eece7fbfa023e22b9fd
5b083acf1d5ebe2d31333f2f2bbf97e4393ff
  Stored in directory: /root/.cache/pip/wheels/2b/4d/8f/55fb4f7b9b591891e8d3f72977c4ec6c7763b39c19f0861595
Successfully built GPUtil
Installing collected packages: GPUtil
Successfully installed GPUtil-1.4.0
```

In [26]:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import psutil
import GPUtil
from itertools import product
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import ParameterGrid
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler, SMOTE
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, co
nfusion_matrix
from google.colab import drive
```

Sambungkan ke Google Drive dan arahkan ke path data yang akan digunakan

In [2]:

```
drive.mount('/content/drive')

path_data = "/content/drive/MyDrive/Bengkel Coding/Pertemuan 4/Dataset/adult.csv"
```

```
Mounted at /content/drive
```

Baca dataset

Pada data yang kita gunakan kali ini, tantangannya adalah bahwa data ini masih belum memiliki nama kolom, maka nanti akan kita beri nama pada kolomnya termasuk akan kita analisis mengenai langkah apa saja yang akan kita gunakan dalam melakukan pre-processing data

In [3]:

```
df = pd.read_csv(path_data)
df.head()
```

Out[3]:

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United-States	<=50K

In [4]:

```
df.tail()
```

Out[4]:

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
32555	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support	Wife	White	Female	0	0	38	United-States	<=50K
32556	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	0	0	40	United-States	>50K
32557	58	Private	151910	HS-grad	9	Widowed	Adm-clerical	Unmarried	White	Female	0	0	40	United-States	<=50K
32558	22	Private	201490	HS-grad	9	Never-married	Adm-clerical	Own-child	White	Male	0	0	20	United-States	<=50K
32559	52	Self-emp-inc	287927	HS-grad	9	Married-civ-spouse	Exec-managerial	Wife	White	Female	15024	0	40	United-States	>50K

2. Pre-Processing Data

Memberikan nama kolom pada data

In [5]:

```
col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status',
             'occupation', 'relationship', 'race', 'sex', 'capital_gain', 'capital_loss',
             'hours_per_week', 'native_country', 'income']
```

In [6]:

```
df.columns = col_names
df.head()
```

Out[6]:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	

In [7]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32560 entries, 0 to 32559
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32560 non-null  int64
 1   workclass       32560 non-null  object
 2   fnlwgt          32560 non-null  int64
 3   education       32560 non-null  object
 4   education_num   32560 non-null  int64
 5   marital_status  32560 non-null  object
 6   occupation      32560 non-null  object
 7   relationship    32560 non-null  object
 8   race            32560 non-null  object
 9   sex             32560 non-null  object
 10  capital_gain    32560 non-null  int64
 11  capital_loss    32560 non-null  int64
 12  hours_per_week  32560 non-null  int64
 13  native_country  32560 non-null  object
 14  income          32560 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

tidak ditemukan data yang hilang karena jumlah seluruh value sama.

In [8]:

```
df.describe()
```

Out[8]:

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week
count	32560.000000	3.256000e+04	32560.000000	32560.000000	32560.000000	32560.000000
mean	38.581634	1.897818e+05	10.080590	1077.615172	87.306511	40.437469

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week
std	13.640642	1.055498e+05	2.572709	7385.402999	402.966116	12.347618
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.178315e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.783630e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.370545e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	99.000000

Menghitung nilai unik pada masing-masing class

Di sini kita akan melakukan pengecekan terkait nilai-nilai unik yang ada pada tiap-tiap kolom yang bertipe kategorikal

In [9]:

```
unique_counts = df.nunique()
print(unique_counts)
```

```
age                 73
workclass            9
fnlwgt           21647
education           16
education_num       16
marital_status       7
occupation          15
relationship         6
race                 5
sex                  2
capital_gain       119
capital_loss        92
hours_per_week      94
native_country      42
income               2
dtype: int64
```

di sini kita melakukan pengecekan value terhadap seluruh kolom yang berisi string

In [10]:

```
kolom = ["workclass", "education", "marital_status", "occupation", "relationship", "race", "sex", "native_cou
ntry", "income"]
for col in kolom:
    print(f"Kolom: {col}")
    print(df[col].unique())
    print("-" * 40)
```

```
Kolom: workclass
[' Self-emp-not-inc' ' Private' ' State-gov' ' Federal-gov' ' Local-gov'
 ' ?' ' Self-emp-inc' ' Without-pay' ' Never-worked']
----------------------------------------
Kolom: education
[' Bachelors' ' HS-grad' ' 11th' ' Masters' ' 9th' ' Some-college'
 ' Assoc-acdm' ' Assoc-voc' ' 7th-8th' ' Doctorate' ' Prof-school'
 ' 5th-6th' ' 10th' ' 1st-4th' ' Preschool' ' 12th']
----------------------------------------
Kolom: marital_status
[' Married-civ-spouse' ' Divorced' ' Married-spouse-absent'
 ' Never-married' ' Separated' ' Married-AF-spouse' ' Widowed']
----------------------------------------
Kolom: occupation
[' Exec-managerial' ' Handlers-cleaners' ' Prof-specialty'
 ' Other-service' ' Adm-clerical' ' Sales' ' Craft-repair'
 ' Transport-moving' ' Farming-fishing' ' Machine-op-inspct'
 ' Tech-support' ' ?' ' Protective-serv' ' Armed-Forces'
 ' Priv-house-serv']
----------------------------------------
Kolom: relationship
[' Husband' ' Not-in-family' ' Wife' ' Own-child' ' Unmarried'
 ' Other-relative']
----------------------------------------
Kolom: race
[' White' ' Black' ' Asian-Pac-Islander' ' Amer-Indian-Eskimo' ' Other']
----------------------------------------
Kolom: sex
[' Male' ' Female']
----------------------------------------
Kolom: native_country
[' United-States' ' Cuba' ' Jamaica' ' India' ' ?' ' Mexico' ' South'
 ' Puerto-Rico' ' Honduras' ' England' ' Canada' ' Germany' ' Iran'
 ' Philippines' ' Italy' ' Poland' ' Columbia' ' Cambodia' ' Thailand'
 ' Ecuador' ' Laos' ' Taiwan' ' Haiti' ' Portugal' ' Dominican-Republic'
 ' El-Salvador' ' France' ' Guatemala' ' China' ' Japan' ' Yugoslavia'
 ' Peru' ' Outlying-US(Guam-USVI-etc)' ' Scotland' ' Trinadad&Tobago'
 ' Greece' ' Nicaragua' ' Vietnam' ' Hong' ' Ireland' ' Hungary'
 ' Holand-Netherlands']
----------------------------------------
Kolom: income
[' <=50K' ' >50K']
----------------------------------------
```

karena ada beberapa record yang memiliki value "?" maka akan kita ganti dengan "NaN" kemudian akan kita ganti dengan data mayoritas untuk data bertipe kategorikal, dan mean untuk data yang bertipe numerik

In [11]:

```python
# Class untuk Preprocessing Data
class DataPreprocessor:
    def __init__(self, df):
        self.df = df.copy()
        self.df_cleaned = None
        self.fill_values = {}

    def clean_whitespace(self):
        self.df = self.df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

    def replace_missing_values(self):
        self.df.replace("?", np.nan, inplace=True)

    def fill_missing_values(self):
        for col in self.df.columns:
            if self.df[col].dtype == "object":
                if not self.df[col].mode().empty:
                    mode_value = self.df[col].mode().iloc[0]
                    self.df[col].fillna(mode_value, inplace=True)
                    self.fill_values[col] = mode_value
            else:
                mean_value = self.df[col].astype(float).mean()
                self.df[col].fillna(mean_value, inplace=True)
                self.fill_values[col] = mean_value

    def remove_rows_with_missing_values(self):
        self.df_cleaned = self.df.dropna()

    def preprocess(self):
        self.clean_whitespace()
        self.replace_missing_values()
        self.fill_missing_values()
        self.remove_rows_with_missing_values()

    def get_processed_data(self):
        return self.df_cleaned if self.df_cleaned is not None else self.df

# Preprocess Data
preprocessor = DataPreprocessor(df)
preprocessor.preprocess()
df = preprocessor.get_processed_data()
```

```
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mode_value, inplace=True)
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mode_value, inplace=True)
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
  self.df[col].fillna(mode_value, inplace=True)
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mean_value, inplace=True)
<ipython-input-11-42455891a13d>:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
es through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on
which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)
' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  self.df[col].fillna(mode_value, inplace=True)
```

In [12]:

```
kolom = ["workclass", "education", "marital_status", "occupation", "relationship", "race", "sex", "native_cou
ntry", "income"]
for col in kolom:
    print(f"Kolom: {col}")
    print(df[col].unique())
    print("-" * 40)
```

```
Kolom: workclass
['Self-emp-not-inc' 'Private' 'State-gov' 'Federal-gov' 'Local-gov'
 'Self-emp-inc' 'Without-pay' 'Never-worked']
----------------------------------------
Kolom: education
['Bachelors' 'HS-grad' '11th' 'Masters' '9th' 'Some-college' 'Assoc-acdm'
 'Assoc-voc' '7th-8th' 'Doctorate' 'Prof-school' '5th-6th' '10th'
 '1st-4th' 'Preschool' '12th']
----------------------------------------
Kolom: marital_status
['Married-civ-spouse' 'Divorced' 'Married-spouse-absent' 'Never-married'
 'Separated' 'Married-AF-spouse' 'Widowed']
----------------------------------------
Kolom: occupation
['Exec-managerial' 'Handlers-cleaners' 'Prof-specialty' 'Other-service'
 'Adm-clerical' 'Sales' 'Craft-repair' 'Transport-moving'
 'Farming-fishing' 'Machine-op-inspct' 'Tech-support' 'Protective-serv'
 'Armed-Forces' 'Priv-house-serv']
----------------------------------------
Kolom: relationship
['Husband' 'Not-in-family' 'Wife' 'Own-child' 'Unmarried' 'Other-relative']
----------------------------------------
Kolom: race
['White' 'Black' 'Asian-Pac-Islander' 'Amer-Indian-Eskimo' 'Other']
----------------------------------------
Kolom: sex
['Male' 'Female']
----------------------------------------
Kolom: native_country
['United-States' 'Cuba' 'Jamaica' 'India' 'Mexico' 'South' 'Puerto-Rico'
 'Honduras' 'England' 'Canada' 'Germany' 'Iran' 'Philippines' 'Italy'
 'Poland' 'Columbia' 'Cambodia' 'Thailand' 'Ecuador' 'Laos' 'Taiwan'
 'Haiti' 'Portugal' 'Dominican-Republic' 'El-Salvador' 'France'
 'Guatemala' 'China' 'Japan' 'Yugoslavia' 'Peru'
 'Outlying-US(Guam-USVI-etc)' 'Scotland' 'Trinadad&Tobago' 'Greece'
 'Nicaragua' 'Vietnam' 'Hong' 'Ireland' 'Hungary' 'Holand-Netherlands']
----------------------------------------
Kolom: income
['<=50K' '>50K']
----------------------------------------
```

Karena data dengan tipe kategorikal bentuknya masih berupa object dan belum berupa angka, maka selanjutnya kita perlu untuk mengubah agar tipe dari record yang bertipe object menjadi bertipe angka, kita akan menggunakan label encoder dari pustaka sklearn

In [13]:

```
from sklearn.preprocessing import LabelEncoder
```

```
kolom = ["workclass", "education", "marital_status", "occupation", "relationship", "race", "sex", "native_cou
ntry", "income"]

le = LabelEncoder()

for col in kolom:
    df[col] = le.fit_transform(df[col])
df.head()
```

Out[13]:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week
0	50	5	83311	9	13	2	3	0	4	1	0	0	13
1	38	3	215646	11	9	0	5	1	4	1	0	0	40
2	53	3	234721	1	7	2	5	0	2	1	0	0	40
3	28	3	338409	9	13	2	9	5	2	0	0	0	40
4	37	3	284582	12	14	2	3	5	4	0	0	0	40

untuk kolom target (income) ketika penghasilan <= 50k maka diubah ke angka 0, dan jika penghasilan > 50k diubah ke angka 1

Melakukan Pengecekan Missing Value

In [14]:

```
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])
```

```
Series([], dtype: int64)
```

In [15]:

```
df.describe()
```

Out[15]:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	
count	32560.000000	32560.000000	3.256000e+04	32560.000000	32560.000000	32560.000000	32560.000000	32560.000000	32560.000000	32560.00
mean	38.581634	3.094349	1.897818e+05	10.298249	10.080590	2.611794	6.138943	1.446376	3.665848	0.66
std	13.640642	1.107094	1.055498e+05	3.870317	2.572709	1.506225	3.972623	1.606794	0.848817	0.47
min	17.000000	0.000000	1.228500e+04	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.00
25%	28.000000	3.000000	1.178315e+05	9.000000	9.000000	2.000000	3.000000	0.000000	4.000000	0.00
50%	37.000000	3.000000	1.783630e+05	11.000000	10.000000	2.000000	6.000000	1.000000	4.000000	1.00
75%	48.000000	3.000000	2.370545e+05	12.000000	12.000000	4.000000	9.000000	3.000000	4.000000	1.00
max	90.000000	7.000000	1.484705e+06	15.000000	16.000000	6.000000	13.000000	5.000000	4.000000	1.00

3. Pemodelan

di sini kita akan menggunakan model Random Forest, kenapa Random Forest?, karena pada hasil pengujian yang telah saya lakukan, Random Forest yang berhasil mendapatkan akurasi tertinggi dibandingkan dengan model-model lainnya

In [16]:

```
# pisahkan fitur dan target
X = df.drop("income", axis=1)
y = df["income"]
```

In [17]:

```
# bagi data menjadi train-test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

In [18]:

```
# normalisasi fitur numerik
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [19]:

```
# Fungsi evaluasi model dengan waktu
def evaluate_model(model, X_train, X_test, y_train, y_test):
    start = time.time()
    model.fit(X_train, y_train)
    end = time.time()
    y_pred = model.predict(X_test)
    training_time = end - start
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
    report = classification_report(y_test, y_pred, zero_division=0)
    cm = confusion_matrix(y_test, y_pred)
    return accuracy, precision, recall, f1, training_time, report, cm
```

di sini kami langsung menggunakan empat sampling methods untuk menguji kira-kira dengan tipe handling data imbalance mana akurasi terbaik akan didapatkan

In [20]:

```
# Sampling
sampling_methods = {
    "No Sampling": None,
    "Undersampling": RandomUnderSampler(random_state=42),
    "Oversampling": RandomOverSampler(random_state=42),
    "SMOTE": SMOTE(random_state=42)
}
```

kami menguji menggunakan model Random Forest dan SVM

In [21]:

```
# Model
models = {
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "KNN": KNeighborsClassifier(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "SVM": SVC()
}
```

parameter yang dicantumkan di sini adalah parameter terbaik yang telah kami uji sebelumnya dengan hyperparameter tuning, sengaja kami cantumkan yang terbaik karena kalau semua parameter kami cantumkan maka tidak efisien pada waktu eksekusi nantinya

In [22]:

```
# Hyperparameter
param_grid = {
    "Decision Tree": {
        "max_depth": [10, 15]
    },
    "Random Forest": {
        "n_estimators": [200],
        "max_depth": [15],
        "min_samples_split": [5],
        "min_samples_leaf": [1],
        "max_features": [None]
    },
    "KNN": {
        "n_neighbors": [3, 5]
    },
    "Logistic Regression": {
        "C": [1.0]
    },
    "SVM": {
        "C": [1],
        "kernel": ["rbf"],
        "gamma": ["scale"]
    }
}
```

Pada bagian ini kita menambahkan fungsi baru untuk melakukan tracking penggunaan CPU dan GPU selama pelatihan data

In [27]:

```
result = {}
conf_matrices = {}

def get_system_usage():
    cpu_usage = psutil.cpu_percent(interval=1)
    mem = psutil.virtual_memory().percent
    try:
        gpus = GPUtil.getGPUs()
        if gpus:
            gpu_usage = gpus[0].load * 100  # ambil GPU pertama
        else:
            gpu_usage = None
    except:
        gpu_usage = None
    return cpu_usage, mem, gpu_usage
```

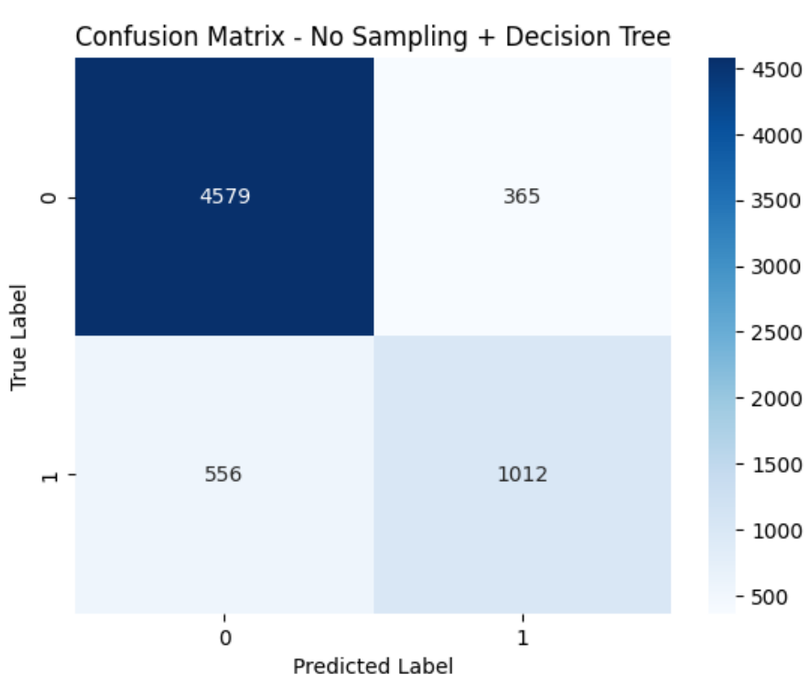
Pada bagian ini data dilatih, pelatihan menggunakan sistem looping untuk menguji kemungkinan-kemungkinan kombinasi parameter (Hyperparameter) yang nantinya diharapkan mampu memberikan akurasi terbaik, selanjutnya akan ditampilkan juga hasil evaluasi, penggunaan CPU dan GPU, serta Conf Matrix dari tiap-tiap kombinasi parameternya

In [38]:

```
for sampling_name, sampler in sampling_methods.items():
    print(f"\nEvaluating models with {sampling_name}")
    if sampler:
        X_resampled, y_resampled = sampler.fit_resample(X_train, y_train)
    else:
        X_resampled, y_resampled = X_train, y_train

    for name, model in models.items():
```

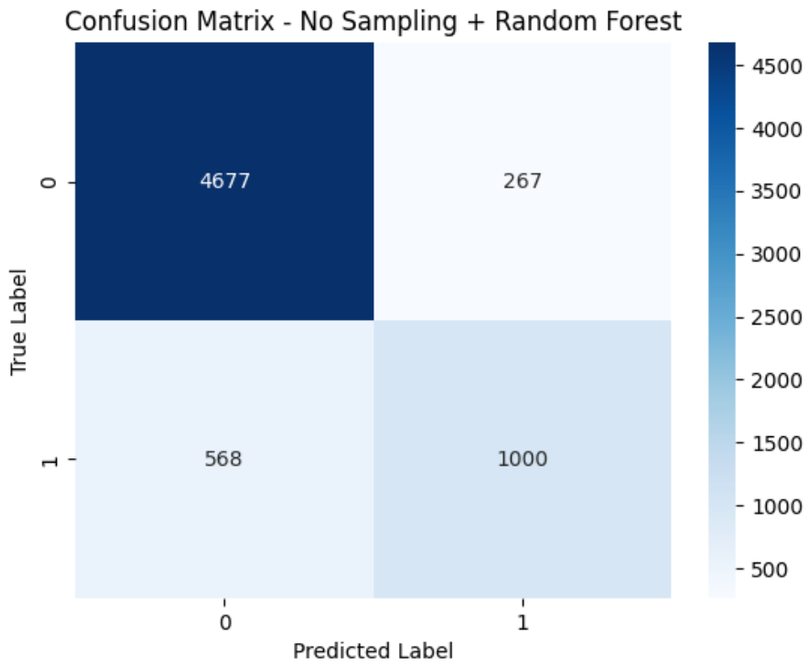
```
        best_params = {}

        if name in param_grid:
            grid_search = GridSearchCV(model, param_grid[name], cv=5, scoring="accuracy", n_jobs=-1)
            grid_search.fit(X_resampled, y_resampled)
            model = grid_search.best_estimator_
            best_params = grid_search.best_params_

        # Monitor resource usage
        cpu_usage, mem_usage, gpu_usage = get_system_usage()

        accuracy, precision, recall, f1, train_time, report, cm = evaluate_model(
            model, X_resampled, X_test, y_resampled, y_test
        )

        result_key = (sampling_name, name)
        result[result_key] = {
            'Best Parameters': best_params,
            'Accuracy': accuracy,
            'Precision': precision,
            'Recall': recall,
            'F1 Score': f1,
            'Training Time': train_time,
            'Classification Report': report,
            'CPU Usage (%)': cpu_usage,
            'Memory Usage (%)': mem_usage,
            'GPU Usage (%)': gpu_usage if gpu_usage is not None else "N/A"
        }
        conf_matrices[result_key] = cm

        # Langsung tampilkan hasil evaluasi setelah training
        print(f"\n--- Evaluation: {sampling_name} + {name} ---")
        print(f"Best Parameters: {best_params}")
        print(f"Accuracy: {accuracy:.4f}")
        print(f"Precision: {precision:.4f}")
        print(f"Recall: {recall:.4f}")
        print(f"F1 Score: {f1:.4f}")
        print(f"Training Time: {train_time:.4f} seconds")
        print(f"CPU Usage: {cpu_usage}% | Memory Usage: {mem_usage}% | GPU Usage: {gpu_usage if gpu_usage is n
ot None else 'N/A'}")
        print(f"Classification Report:\n{report}")
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
        plt.title(f"Confusion Matrix - {sampling_name} + {name}")
        plt.xlabel("Predicted Label")
        plt.ylabel("True Label")
        plt.show()
        print("=" * 100)
```

```
Evaluating models with No Sampling

--- Evaluation: No Sampling + Decision Tree ---
Best Parameters: {'max_depth': 10}
Accuracy: 0.8586
Precision: 0.8540
Recall: 0.8586
F1 Score: 0.8553
Training Time: 0.3525 seconds
CPU Usage: 67.3% | Memory Usage: 15.9% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.93      0.91      4944
           1       0.73      0.65      0.69      1568

    accuracy                           0.86      6512
   macro avg       0.81      0.79      0.80      6512
weighted avg       0.85      0.86      0.86      6512
```

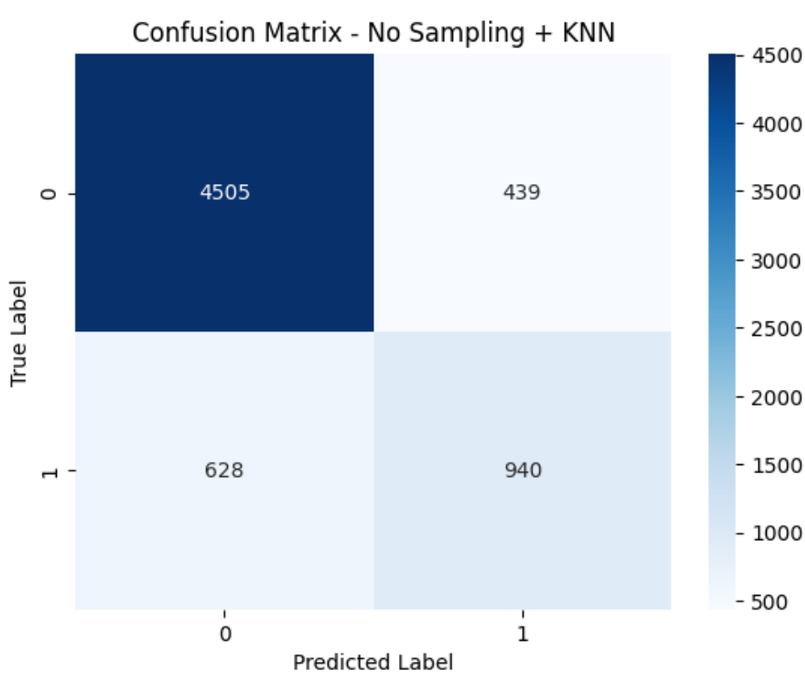



```
====================================================================================================

--- Evaluation: No Sampling + Random Forest ---
```

```
Best Parameters: {'max_depth': 15, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_esti
mators': 200}
Accuracy: 0.8718
Precision: 0.8670
Recall: 0.8718
F1 Score: 0.8669
Training Time: 20.9955 seconds
CPU Usage: 2.0% | Memory Usage: 16.6% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.95      0.92      4944
           1       0.79      0.64      0.71      1568

    accuracy                           0.87      6512
   macro avg       0.84      0.79      0.81      6512
weighted avg       0.87      0.87      0.87      6512
```

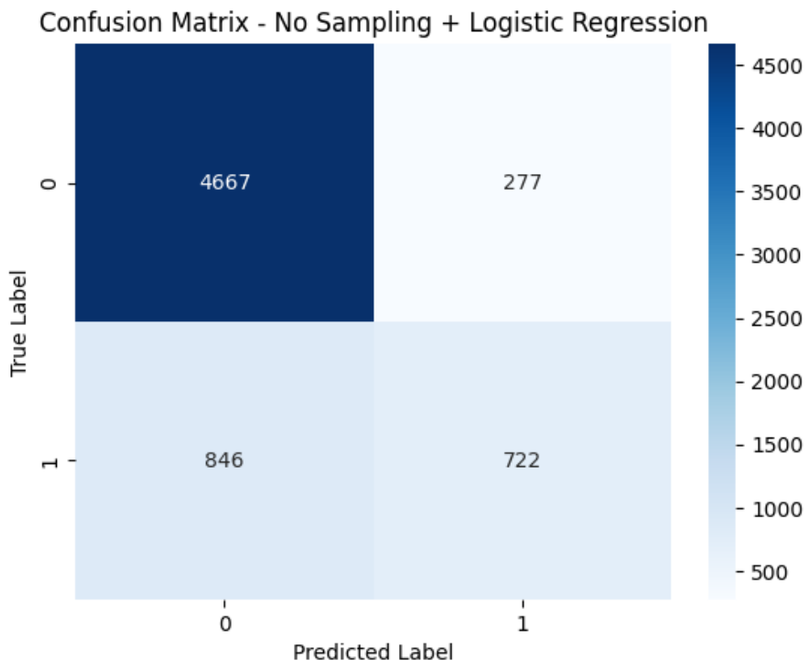



```
==========================================================================================

--- Evaluation: No Sampling + KNN ---
Best Parameters: {'n_neighbors': 5}
Accuracy: 0.8361
Precision: 0.8305
Recall: 0.8361
F1 Score: 0.8324
Training Time: 0.0554 seconds
CPU Usage: 4.5% | Memory Usage: 16.1% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.91      0.89      4944
           1       0.68      0.60      0.64      1568

    accuracy                           0.84      6512
   macro avg       0.78      0.76      0.77      6512
weighted avg       0.83      0.84      0.83      6512
```

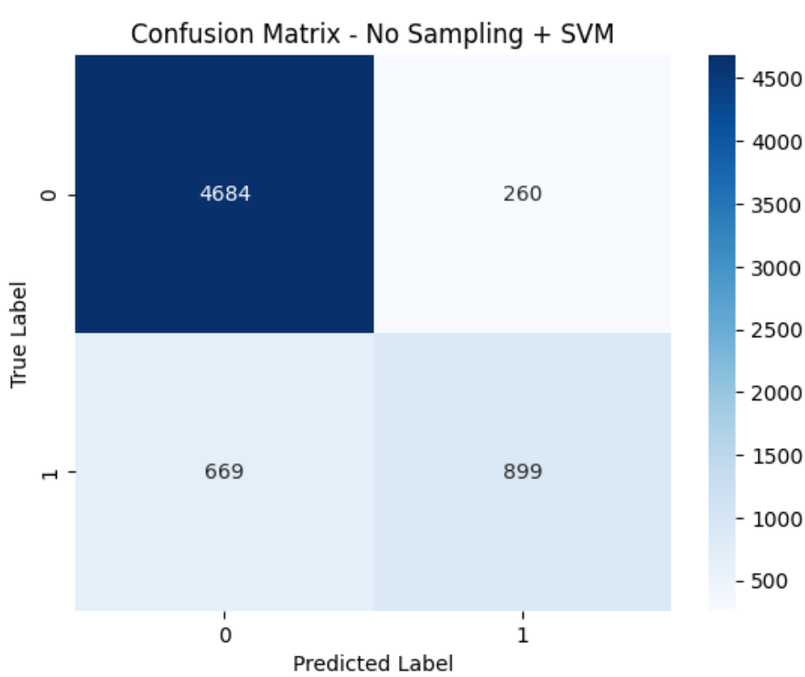



```
==========================================================================================

--- Evaluation: No Sampling + Logistic Regression ---
Best Parameters: {'C': 1.0}
```

```
Best Parameters: {'C': 1.0}
Accuracy: 0.8275
Precision: 0.8167
Recall: 0.8275
F1 Score: 0.8131
Training Time: 0.0268 seconds
CPU Usage: 6.0% | Memory Usage: 16.5% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.94      0.89      4944
           1       0.72      0.46      0.56      1568

    accuracy                           0.83      6512
   macro avg       0.78      0.70      0.73      6512
weighted avg       0.82      0.83      0.81      6512
```

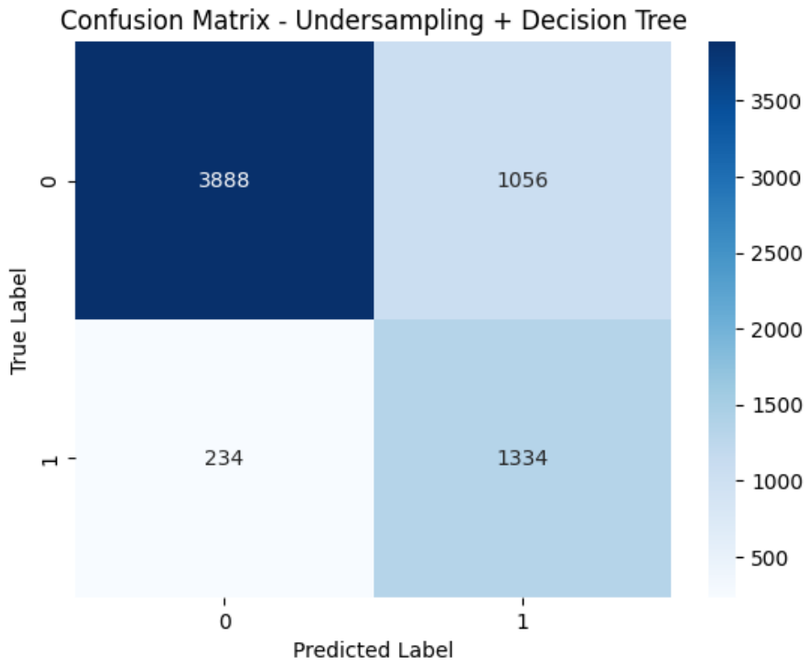



```
==========================================================================================

--- Evaluation: No Sampling + SVM ---
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.8573
Precision: 0.8511
Recall: 0.8573
F1 Score: 0.8495
Training Time: 17.8191 seconds
CPU Usage: 2.5% | Memory Usage: 16.3% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.95      0.91      4944
           1       0.78      0.57      0.66      1568

    accuracy                           0.86      6512
   macro avg       0.83      0.76      0.78      6512
weighted avg       0.85      0.86      0.85      6512
```

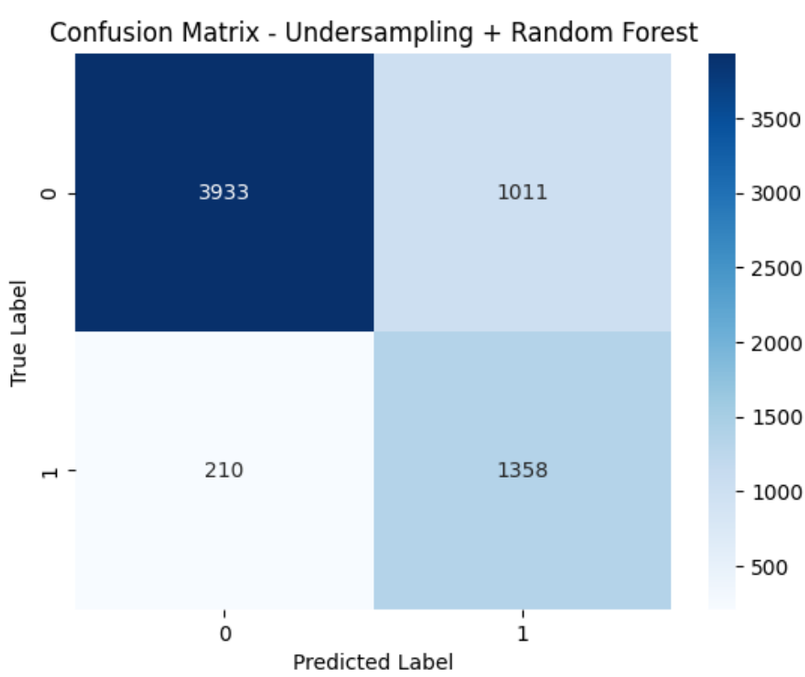



```
==========================================================================================

Evaluating models with Undersampling

--- Evaluation: Undersampling + Decision Tree ---
```

```
Best Parameters: {'max_depth': 10}
Accuracy: 0.8019
Precision: 0.8505
Recall: 0.8019
F1 Score: 0.8135
Training Time: 0.0446 seconds
CPU Usage: 14.6% | Memory Usage: 16.5% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.79      0.86      4944
           1       0.56      0.85      0.67      1568

    accuracy                           0.80      6512
   macro avg       0.75      0.82      0.77      6512
weighted avg       0.85      0.80      0.81      6512
```

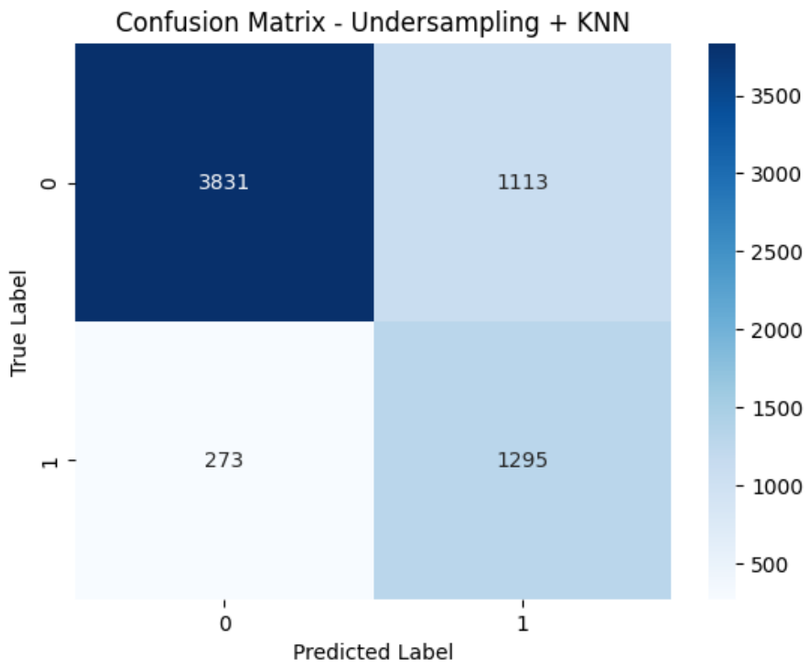



```
==========================================================================================

--- Evaluation: Undersampling + Random Forest ---
Best Parameters: {'max_depth': 15, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_esti
mators': 200}
Accuracy: 0.8125
Precision: 0.8588
Recall: 0.8125
F1 Score: 0.8233
Training Time: 11.1827 seconds
CPU Usage: 4.0% | Memory Usage: 16.3% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.80      0.87      4944
           1       0.57      0.87      0.69      1568

    accuracy                           0.81      6512
   macro avg       0.76      0.83      0.78      6512
weighted avg       0.86      0.81      0.82      6512
```

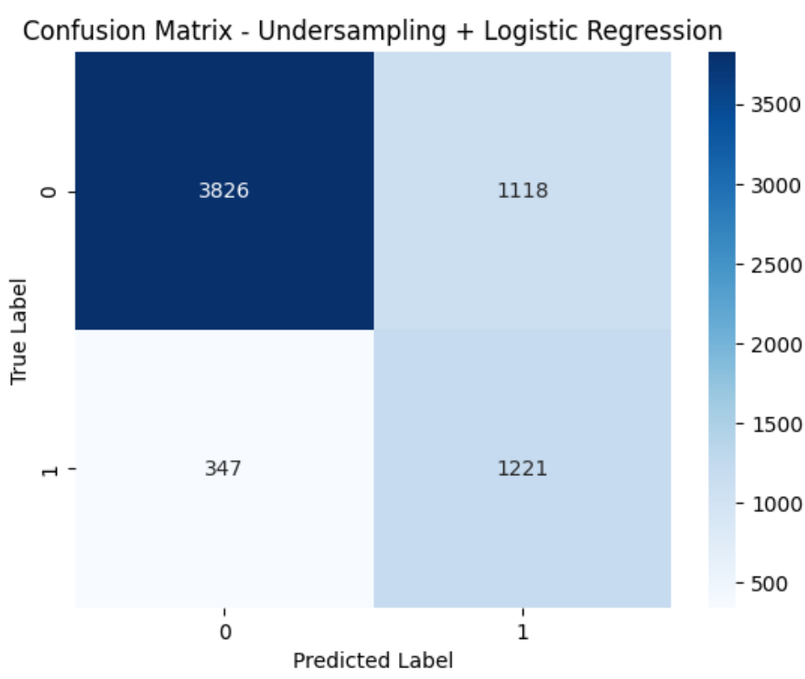



```
==========================================================================================

--- Evaluation: Undersampling + KNN ---
Best Parameters: {'n_neighbors': 5}
```

```
Best Parameters: {'n_neighbors': 5}
Accuracy: 0.7872
Precision: 0.8382
Recall: 0.7872
F1 Score: 0.7998
Training Time: 0.0362 seconds
CPU Usage: 57.8% | Memory Usage: 16.3% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.77      0.85      4944
           1       0.54      0.83      0.65      1568

    accuracy                           0.79      6512
   macro avg       0.74      0.80      0.75      6512
weighted avg       0.84      0.79      0.80      6512
```

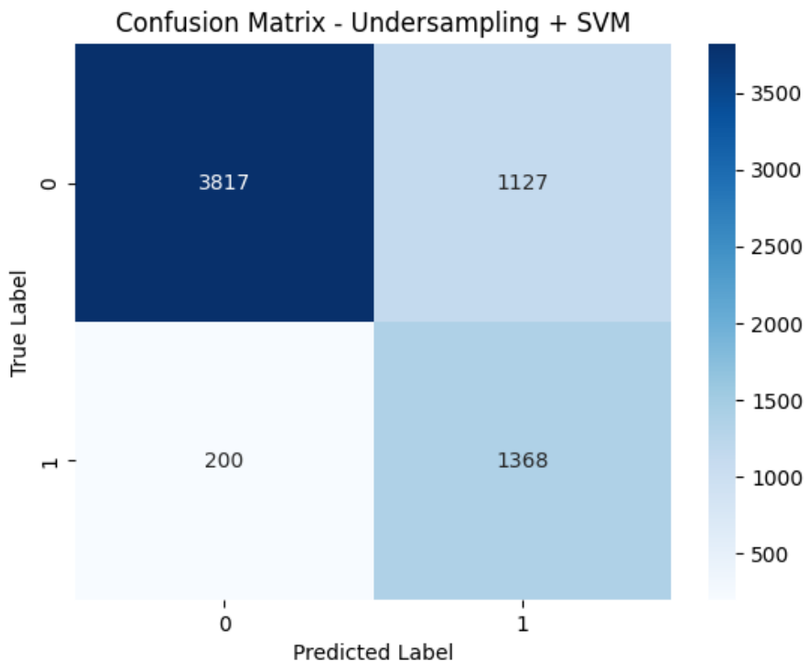



```
==========================================================================================

--- Evaluation: Undersampling + Logistic Regression ---
Best Parameters: {'C': 1.0}
Accuracy: 0.7750
Precision: 0.8218
Recall: 0.7750
F1 Score: 0.7877
Training Time: 0.0160 seconds
CPU Usage: 3.0% | Memory Usage: 16.3% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.77      0.84      4944
           1       0.52      0.78      0.63      1568

    accuracy                           0.78      6512
   macro avg       0.72      0.78      0.73      6512
weighted avg       0.82      0.78      0.79      6512
```

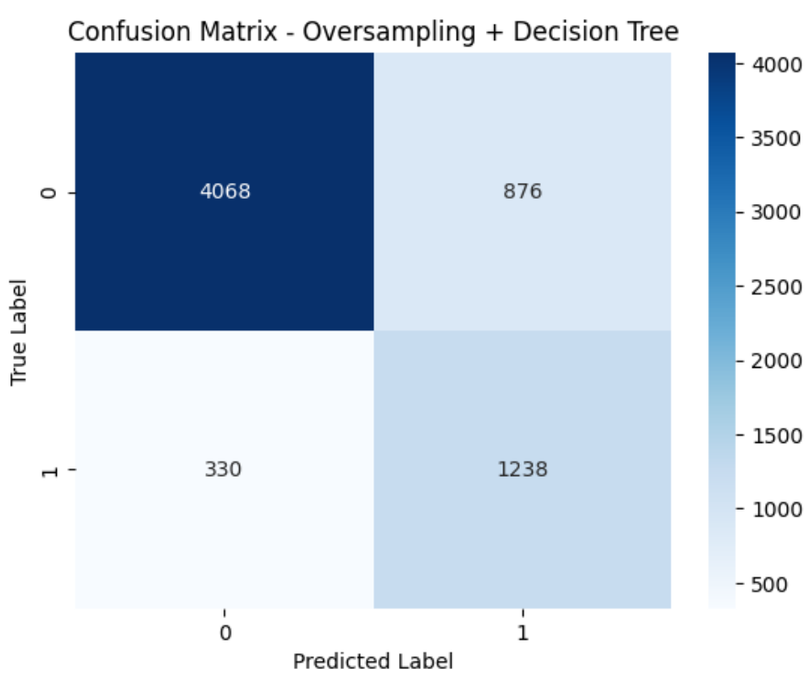



```
==========================================================================================

--- Evaluation: Undersampling + SVM ---
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.7962
```

```
Precision: 0.8534
Recall: 0.7962
F1 Score: 0.8089
Training Time: 7.8961 seconds
CPU Usage: 29.0% | Memory Usage: 16.8% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.77      0.85      4944
           1       0.55      0.87      0.67      1568

    accuracy                           0.80      6512
   macro avg       0.75      0.82      0.76      6512
weighted avg       0.85      0.80      0.81      6512
```

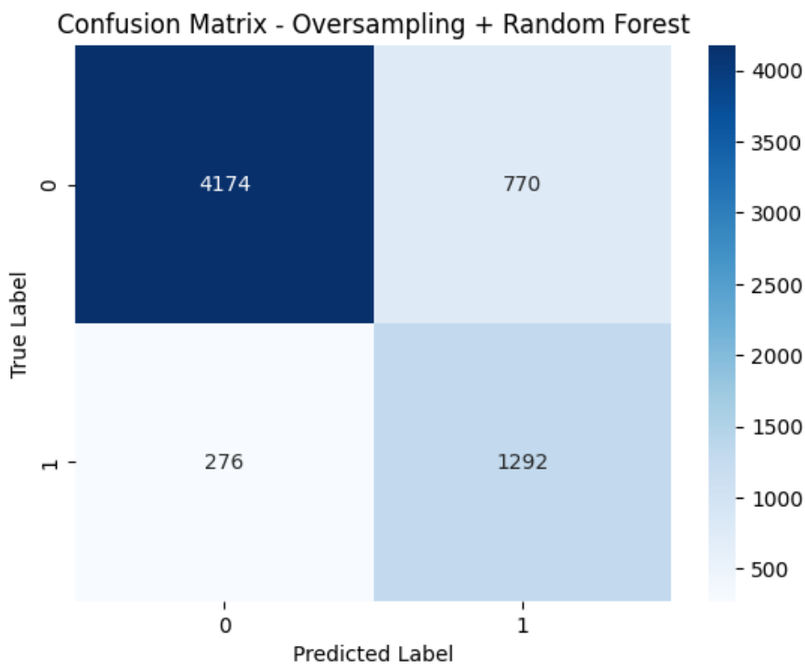



```
==========================================================================================

Evaluating models with Oversampling

--- Evaluation: Oversampling + Decision Tree ---
Best Parameters: {'max_depth': 15}
Accuracy: 0.8148
Precision: 0.8433
Recall: 0.8148
F1 Score: 0.8231
Training Time: 0.3646 seconds
CPU Usage: 51.8% | Memory Usage: 17.2% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.82      0.87      4944
           1       0.59      0.79      0.67      1568

    accuracy                           0.81      6512
   macro avg       0.76      0.81      0.77      6512
weighted avg       0.84      0.81      0.82      6512
```

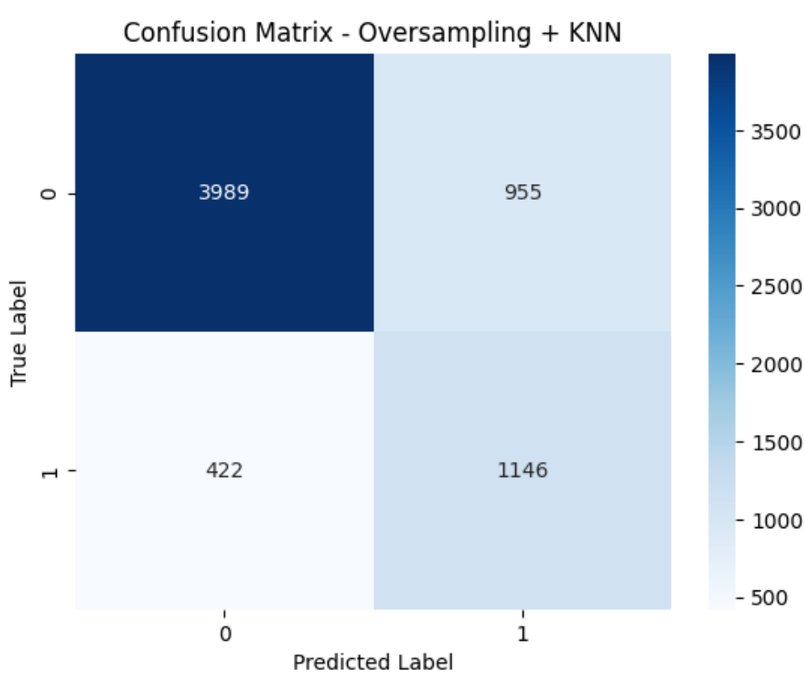



```
==========================================================================================

--- Evaluation: Oversampling + Random Forest ---
Best Parameters: {'max_depth': 15, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_esti
```

```
mators': 200}
Accuracy: 0.8394
Precision: 0.8630
Recall: 0.8394
F1 Score: 0.8461
Training Time: 29.1560 seconds
CPU Usage: 2.5% | Memory Usage: 16.9% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.84      0.89      4944
           1       0.63      0.82      0.71      1568

    accuracy                           0.84      6512
   macro avg       0.78      0.83      0.80      6512
weighted avg       0.86      0.84      0.85      6512
```

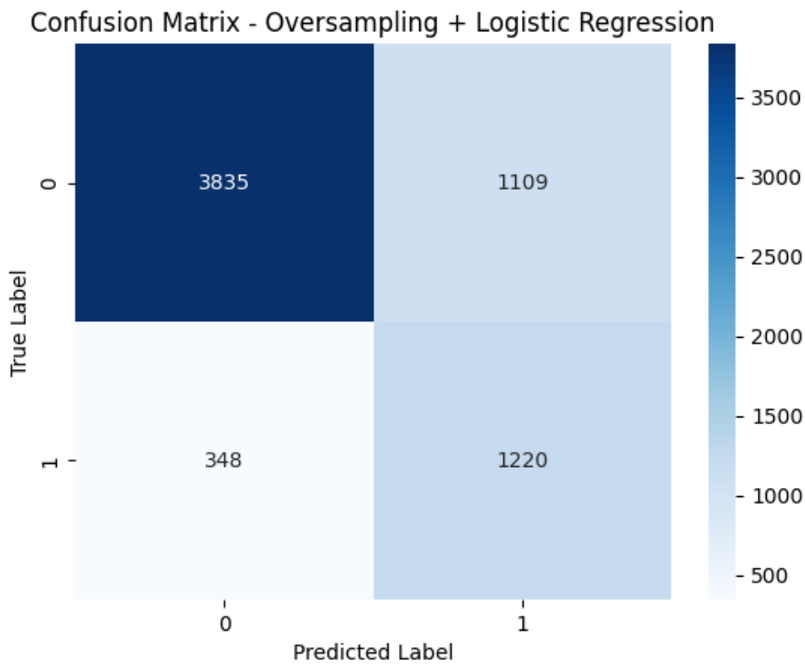



```
==========================================================================================

--- Evaluation: Oversampling + KNN ---
Best Parameters: {'n_neighbors': 3}
Accuracy: 0.7885
Precision: 0.8179
Recall: 0.7885
F1 Score: 0.7979
Training Time: 0.0889 seconds
CPU Usage: 2.5% | Memory Usage: 16.6% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.81      0.85      4944
           1       0.55      0.73      0.62      1568

    accuracy                           0.79      6512
   macro avg       0.72      0.77      0.74      6512
weighted avg       0.82      0.79      0.80      6512
```

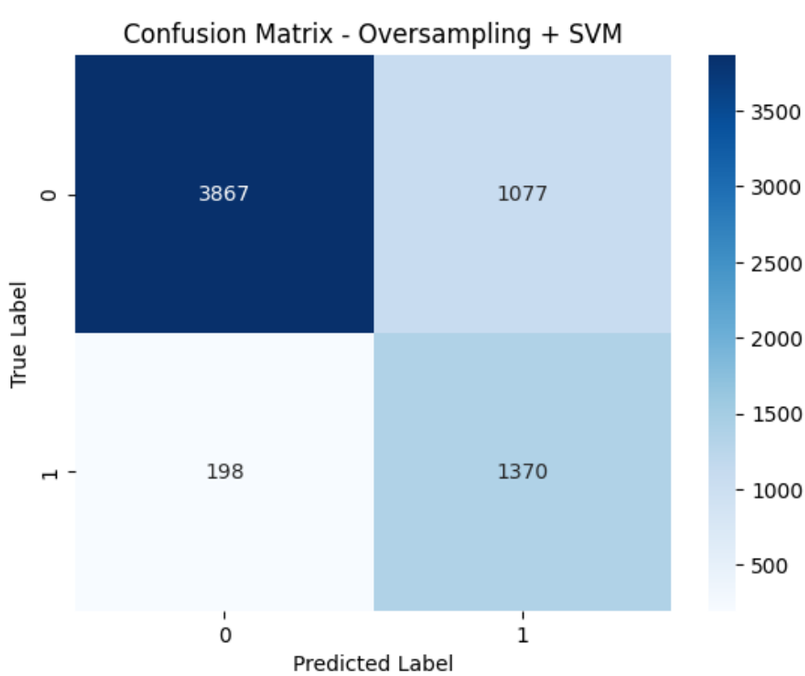



```
==========================================================================================

--- Evaluation: Oversampling + Logistic Regression ---
Best Parameters: {'C': 1.0}
Accuracy: 0.7763
```

```
Precision: 0.8222
Recall: 0.7763
F1 Score: 0.7888
Training Time: 0.0518 seconds
CPU Usage: 9.5% | Memory Usage: 16.6% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.78      0.84      4944
           1       0.52      0.78      0.63      1568

    accuracy                           0.78      6512
   macro avg       0.72      0.78      0.73      6512
weighted avg       0.82      0.78      0.79      6512
```

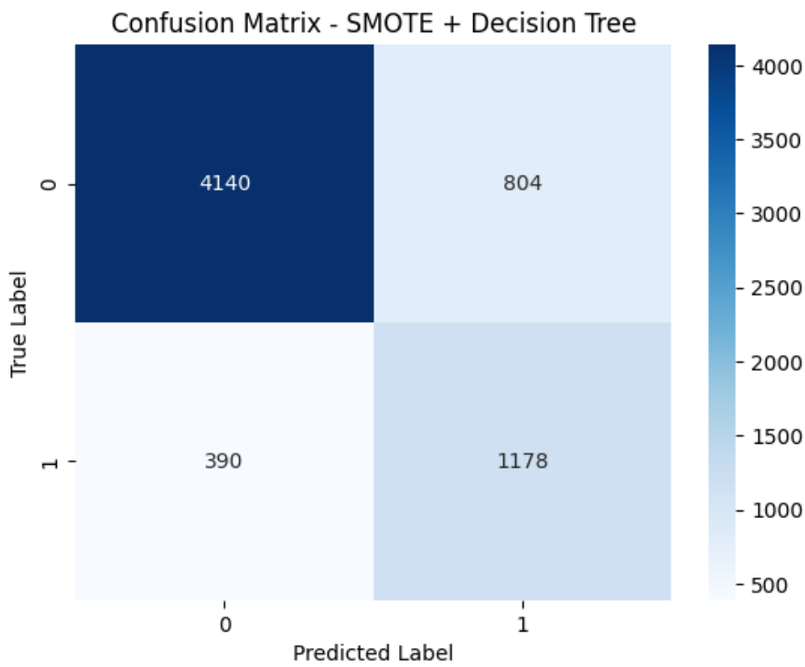



```
==========================================================================================

--- Evaluation: Oversampling + SVM ---
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.8042
Precision: 0.8570
Recall: 0.8042
F1 Score: 0.8161
Training Time: 48.8105 seconds
CPU Usage: 2.5% | Memory Usage: 16.4% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.78      0.86      4944
           1       0.56      0.87      0.68      1568

    accuracy                           0.80      6512
   macro avg       0.76      0.83      0.77      6512
weighted avg       0.86      0.80      0.82      6512
```

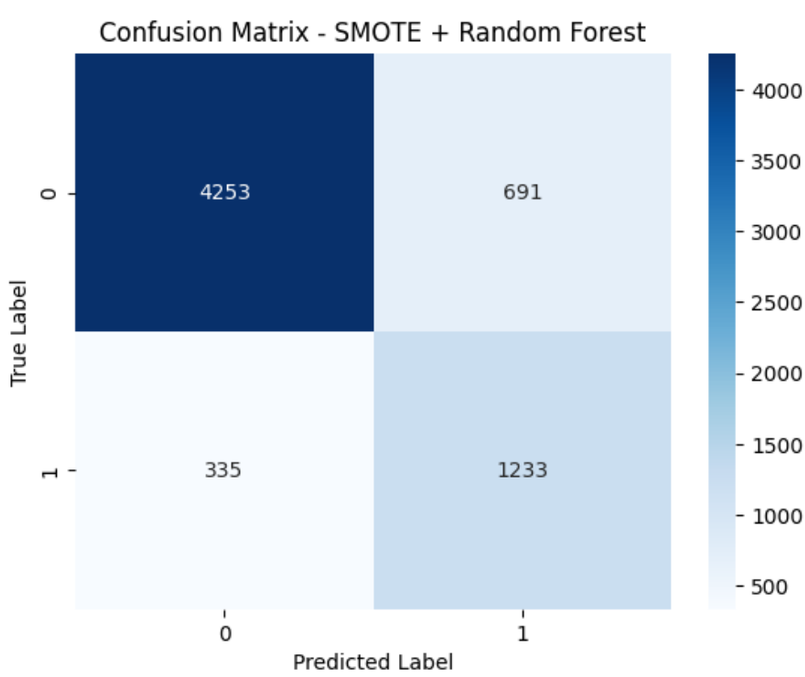



```
==========================================================================================

Evaluating models with SMOTE

--- Evaluation: SMOTE + Decision Tree ---
Best Parameters: {'max_depth': 15}
```

```
Accuracy: 0.8166
Precision: 0.8370
Recall: 0.8166
F1 Score: 0.8233
Training Time: 0.2726 seconds
CPU Usage: 3.0% | Memory Usage: 17.1% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.84      0.87      4944
           1       0.59      0.75      0.66      1568

    accuracy                           0.82      6512
   macro avg       0.75      0.79      0.77      6512
weighted avg       0.84      0.82      0.82      6512
```

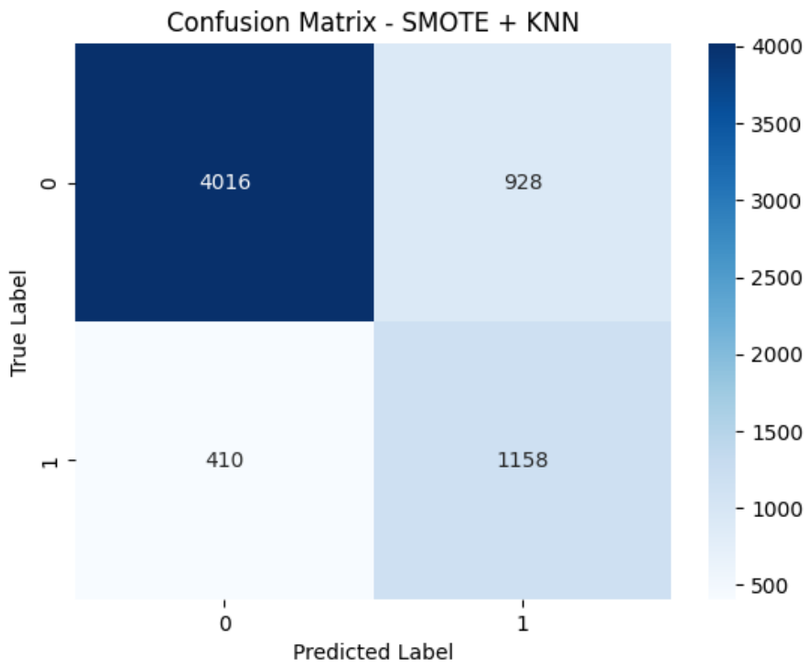



```
==========================================================================================

--- Evaluation: SMOTE + Random Forest ---
Best Parameters: {'max_depth': 15, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_esti
mators': 200}
Accuracy: 0.8424
Precision: 0.8581
Recall: 0.8424
F1 Score: 0.8475
Training Time: 38.4577 seconds
CPU Usage: 56.0% | Memory Usage: 17.1% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.86      0.89      4944
           1       0.64      0.79      0.71      1568

    accuracy                           0.84      6512
   macro avg       0.78      0.82      0.80      6512
weighted avg       0.86      0.84      0.85      6512
```

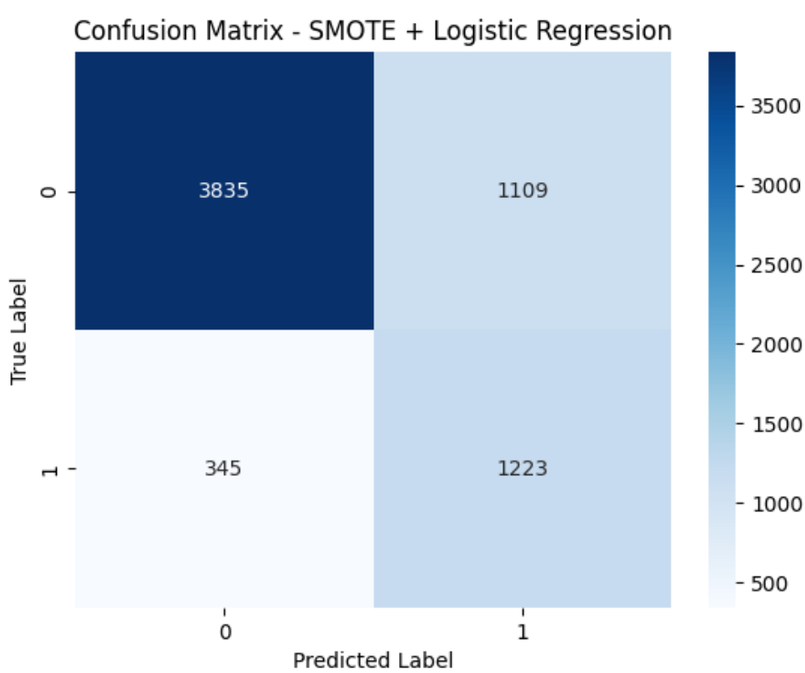



```
==========================================================================================

--- Evaluation: SMOTE + KNN ---
Best Parameters: {'n_neighbors': 3}
Accuracy: 0.7945
```

```
Precision: 0.8226
Recall: 0.7945
F1 Score: 0.8034
Training Time: 0.0888 seconds
CPU Usage: 2.5% | Memory Usage: 17.1% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.81      0.86      4944
           1       0.56      0.74      0.63      1568

    accuracy                           0.79      6512
   macro avg       0.73      0.78      0.75      6512
weighted avg       0.82      0.79      0.80      6512
```

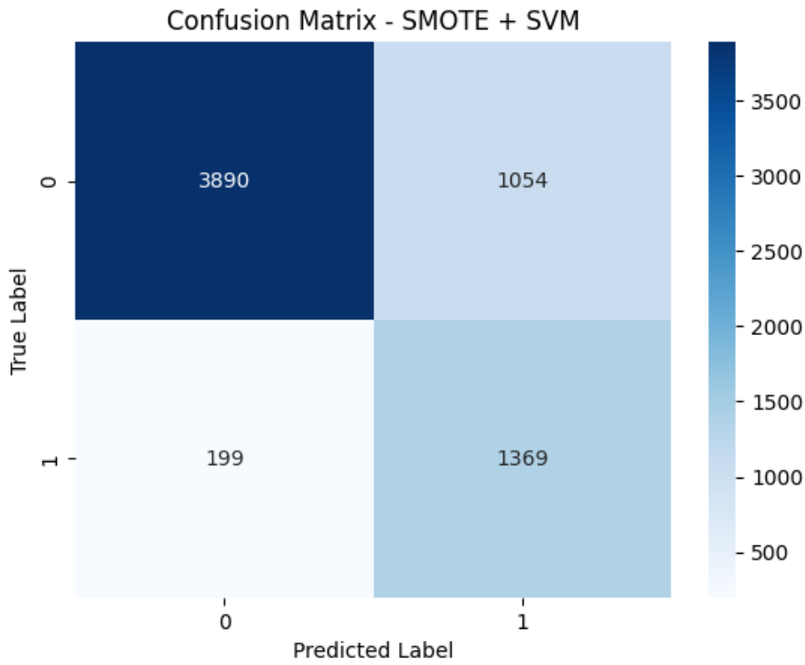



```
==========================================================================================

--- Evaluation: SMOTE + Logistic Regression ---
Best Parameters: {'C': 1.0}
Accuracy: 0.7767
Precision: 0.8228
Recall: 0.7767
F1 Score: 0.7892
Training Time: 0.0493 seconds
CPU Usage: 9.0% | Memory Usage: 17.1% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.78      0.84      4944
           1       0.52      0.78      0.63      1568

    accuracy                           0.78      6512
   macro avg       0.72      0.78      0.73      6512
weighted avg       0.82      0.78      0.79      6512
```




```
==========================================================================================

--- Evaluation: SMOTE + SVM ---
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.8076
Precision: 0.8583
```

```
Recall: 0.8076
F1 Score: 0.8191
Training Time: 45.5468 seconds
CPU Usage: 2.0% | Memory Usage: 16.9% | GPU Usage: N/A
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.79      0.86      4944
           1       0.57      0.87      0.69      1568

    accuracy                           0.81      6512
   macro avg       0.76      0.83      0.77      6512
weighted avg       0.86      0.81      0.82      6512
```



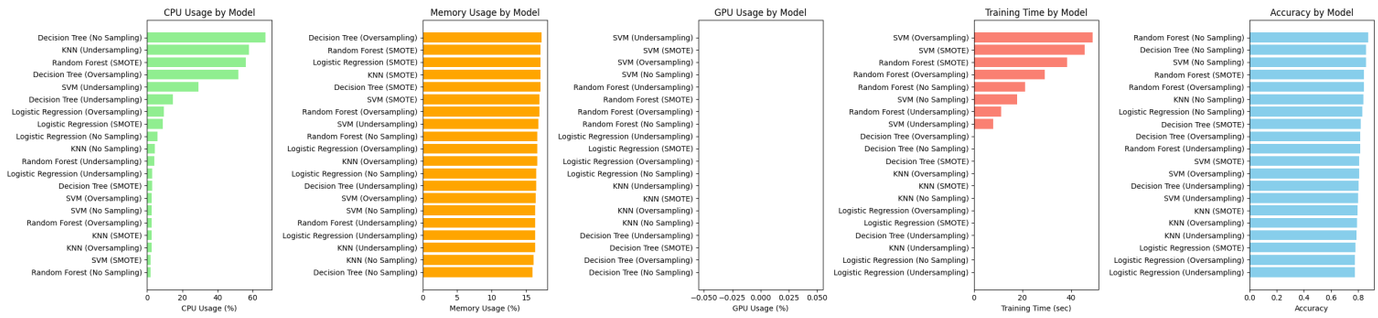
==========

Setelah pelatihan selesai, berikut untuk visualisasi dari hasil yang telah didapatkan

In [40]:

```
# Ambil semua metrik yang dibutuhkan
cpu_usages = [val['CPU Usage (%)'] for val in result.values()]
mem_usages = [val['Memory Usage (%)'] for val in result.values()]
gpu_usages = [
    val['GPU Usage (%)'] if isinstance(val['GPU Usage (%)'], (int, float)) else 0
    for val in result.values()
]
train_times = [val['Training Time'] for val in result.values()]
accuracies = [val['Accuracy'] for val in result.values()]
labels = [f"{model} ({sampling})" for (sampling, model) in result.keys()]

# Sorting masing-masing metrik
cpu_sorted = sorted(zip(cpu_usages, labels))
cpu_usages_sorted, cpu_labels_sorted = zip(*cpu_sorted)

mem_sorted = sorted(zip(mem_usages, labels))
mem_usages_sorted, mem_labels_sorted = zip(*mem_sorted)

gpu_sorted = sorted(zip(gpu_usages, labels))
gpu_usages_sorted, gpu_labels_sorted = zip(*gpu_sorted)

time_sorted = sorted(zip(train_times, labels))
train_times_sorted, time_labels_sorted = zip(*time_sorted)

acc_sorted = sorted(zip(accuracies, labels))
accuracies_sorted, acc_labels_sorted = zip(*acc_sorted)

# Visualisasi semua metrik
fig, axs = plt.subplots(1, 5, figsize=(26, 6))

# CPU Usage
axs[0].barh(cpu_labels_sorted, cpu_usages_sorted, color='lightgreen')
axs[0].set_xlabel("CPU Usage (%)")
axs[0].set_title("CPU Usage by Model")

# Memory Usage
axs[1].barh(mem_labels_sorted, mem_usages_sorted, color='orange')
axs[1].set_xlabel("Memory Usage (%)")
axs[1].set_title("Memory Usage by Model")

# GPU Usage
axs[2].barh(gpu_labels_sorted, gpu_usages_sorted, color='purple')
axs[2].set_xlabel("GPU Usage (%)")
axs[2].set_title("GPU Usage by Model")

# Training Time
axs[3].barh(time_labels_sorted, train_times_sorted, color='salmon')
axs[3].set_xlabel("Training Time (sec)")
axs[3].set_title("Training Time by Model")
```

```
# Accuracy
axs[4].barh(acc_labels_sorted, accuracies_sorted, color='skyblue')
axs[4].set_xlabel("Accuracy")
axs[4].set_title("Accuracy by Model")

plt.tight_layout()
plt.show()
```



Untuk memudahkan dalam membaca detail angkanya, berikut adalah hasilnya:

In [41]:

```
# Top 20 Analisis
top_n = 20

sorted_accuracy = sorted(result.items(), key=lambda x: x[1]['Accuracy'], reverse=True)[:top_n]
sorted_time = sorted(result.items(), key=lambda x: x[1]['Training Time'])[:top_n]
sorted_cpu = sorted(result.items(), key=lambda x: x[1]['CPU Usage (%)'])[:top_n]
sorted_gpu = sorted(
    result.items(),
    key=lambda x: x[1]['GPU Usage (%)'] if isinstance(x[1]['GPU Usage (%)'], (int, float)) else float('inf')
)[:top_n]

print(f"\n=== Top {top_n} Models by Accuracy ===")
for (sampling, model), val in sorted_accuracy:
    print(f"{model} ({sampling}): {val['Accuracy']:.4f}")

print(f"\n=== Top {top_n} Fastest Training Time ===")
for (sampling, model), val in sorted_time:
    print(f"{model} ({sampling}): {val['Training Time']:.4f} sec")

print(f"\n=== Top {top_n} Lowest CPU Usage ===")
for (sampling, model), val in sorted_cpu:
    print(f"{model} ({sampling}): {val['CPU Usage (%)']}%")
```

```
=== Top 20 Models by Accuracy ===
Random Forest (No Sampling): 0.8718
Decision Tree (No Sampling): 0.8586
SVM (No Sampling): 0.8573
Random Forest (SMOTE): 0.8424
Random Forest (Oversampling): 0.8394
KNN (No Sampling): 0.8361
Logistic Regression (No Sampling): 0.8275
Decision Tree (SMOTE): 0.8166
Decision Tree (Oversampling): 0.8148
Random Forest (Undersampling): 0.8125
SVM (SMOTE): 0.8076
SVM (Oversampling): 0.8042
Decision Tree (Undersampling): 0.8019
SVM (Undersampling): 0.7962
KNN (SMOTE): 0.7945
KNN (Oversampling): 0.7885
KNN (Undersampling): 0.7872
Logistic Regression (SMOTE): 0.7767
Logistic Regression (Oversampling): 0.7763
Logistic Regression (Undersampling): 0.7750

=== Top 20 Fastest Training Time ===
Logistic Regression (Undersampling): 0.0160 sec
Logistic Regression (No Sampling): 0.0268 sec
KNN (Undersampling): 0.0362 sec
Decision Tree (Undersampling): 0.0446 sec
Logistic Regression (SMOTE): 0.0493 sec
Logistic Regression (Oversampling): 0.0518 sec
KNN (No Sampling): 0.0554 sec
KNN (SMOTE): 0.0888 sec
KNN (Oversampling): 0.0889 sec
Decision Tree (SMOTE): 0.2726 sec
Decision Tree (No Sampling): 0.3525 sec
Decision Tree (Oversampling): 0.3646 sec
SVM (Undersampling): 7.8961 sec
Random Forest (Undersampling): 11.1827 sec
SVM (No Sampling): 17.8191 sec
Random Forest (No Sampling): 20.9955 sec
Random Forest (Oversampling): 29.1560 sec
Random Forest (SMOTE): 38.4577 sec
SVM (SMOTE): 45.5468 sec
SVM (Oversampling): 48.8105 sec

=== Top 20 Lowest CPU Usage ===
Random Forest (No Sampling): 2.0%
SVM (SMOTE): 2.0%
SVM (No Sampling): 2.5%
Random Forest (Oversampling): 2.5%
KNN (Oversampling): 2.5%
SVM (Oversampling): 2.5%
```

```
KNN (SMOTE): 2.5%
Logistic Regression (Undersampling): 3.0%
Decision Tree (SMOTE): 3.0%
Random Forest (Undersampling): 4.0%
KNN (No Sampling): 4.5%
Logistic Regression (No Sampling): 6.0%
Logistic Regression (SMOTE): 9.0%
Logistic Regression (Oversampling): 9.5%
Decision Tree (Undersampling): 14.6%
SVM (Undersampling): 29.0%
Decision Tree (Oversampling): 51.8%
Random Forest (SMOTE): 56.0%
KNN (Undersampling): 57.8%
Decision Tree (No Sampling): 67.3%
```

Well, dari hasil analisis di atas, kesimpulan yang kami dapatkan adalah bahwa, akurasi tertinggi dari masing-masing model didapatkan dari hasil kombinasi dengan no sampling method. Jadi, mungkin akan ada 2 rekomendasi yang bisa kami sarankan:

1. **Kalau mau akurasi tinggi dan tidak peduli waktu serta penggunaan CPU, kamis sangat merekomendasikan Random Forest (No Sampling)** karena **akurasi yang didapatkan sangat tinggi, menyentuh 0.8718 atau 87.18%.** Selain itu, Random Forest juga punya **keseimbangan precision-recall yang paling baik, terutama di kelas minoritas (class 1) → f1-score: 0.71**, dan **penggunaan CPU yang paling rendah, yaitu sekitar 2.0%.** Namun, yang perlu diperhitungkan adalah **waktu komputasinya yang cukup lama, yaitu 20.9955 sec.**
2. **Kalau mau waktu komputasi rendah tapi masih cukup baik dalam akurasi, kami menyarankan menggunakan Decision Tree (Undersampling)** karena **waktu yang digunakan hanya 0.0446 sec** saja dan akurasi yang didapatkan masih cukup baik, yaitu **0.8586 atau 85.86%**, adapun kekurangannya yaitu **sedikit lebih rendah dalam recall (khususnya untuk class 1)**, serta **penggunaan CPU yang cukup tinggi, menyentuh 67.3%.**

Maka akhirnya mana yang terbaik dan optimal, kami serahkan pada teman-teman sekalian, menyesuaikan dengan kebutuhan masing-masing, hehe:)